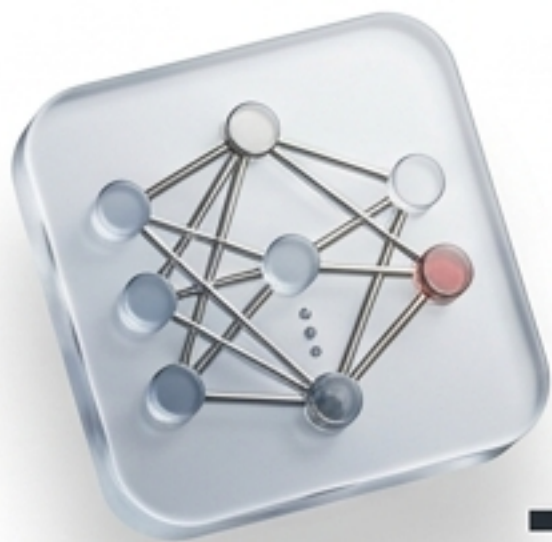
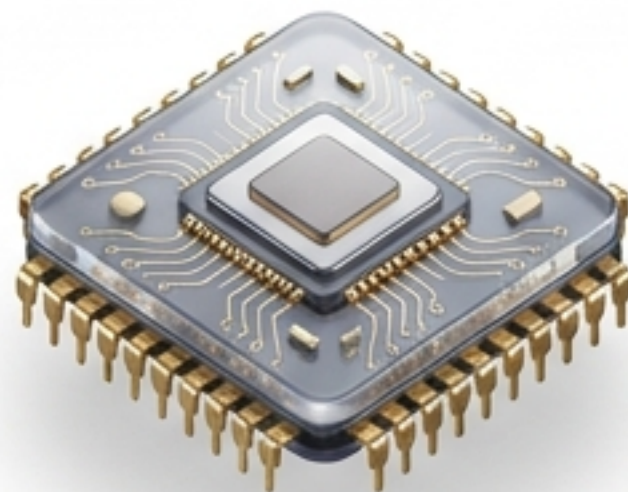




Tối ưu hóa CNN Nâng cao & Giao thời Vision Transformers

Kiến trúc Mini-Xception, Batch Normalization và
và Tương lai của Thị giác Máy tính



Toán học của Batch Normalization

1. Tính Kỳ vọng (μ)



1. Tính Kỳ vọng (μ)

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i$$

Trung bình của Batch

2. Tính Phương sai (σ^2)



2. Tính Phương sai (σ^2)

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$$

Độ phân tán của Batch

3. Chuẩn hóa Gốc



3. Chuẩn hóa Gốc

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Đưa dữ liệu về chuẩn

4. Dịch chuyển & Tỷ lệ



4. Dịch chuyển & Tỷ lệ

$$y_i = \gamma \hat{x}_i + \beta$$

Tham số γ và β học qua Gradient Descent giúp khôi phục thông tin nếu cần.

Quá trình Suy luận & Lời khuyên Transfer Learning



Vấn đề (Inference)

Khi Inference (**Predict**), Batch Size = 1. Không thể tính μ và σ^2 của một ảnh đơn lẻ!



Giải pháp

Lớp BN âm thầm cập nhật Trung bình động lũy thừa (Exponential Moving Average) trong lúc Training.
Khi Inference, nó dùng hằng số Moving Average này thay vì tính lại.



Nguyên tắc Transfer Learning

Luôn đóng băng (Freeze) lớp BN bằng lệnh "trainable=False".

Điều này giúp bảo vệ Moving Average đã được học rất tốt từ ImageNet không bị phá hỏng.

Cách lập trình Batch Normalization chuẩn chuẩn xác



Tối ưu RAM: BN tự động chuẩn hóa Mean về 0, nên tham số 'bias' của lớp Convolution trước đó bị hấp thụ (trở nên vô dụng). Hãy dùng '**use_bias=False**'!



Cách KHÔNG NÊN dùng

Activation bị đặt sai chỗ (trước BN).

```
x = layers.Conv2D(32, 3, activation='relu')(x)
x = layers.BatchNormalization()(x)
```



Cách SỬ DỤNG CHUẨN

BN phải đứng TRƯỚC Hàm kích hoạt.

```
x = layers.Conv2D(32, 3, use_bias=False)(x)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
```

Giới hạn của Tích chập Truyền thống (Conv2D)



Gánh nặng của Tích chập Truyền thống

1. **Học các mẫu Không gian 2D** (Spatial Pattern - VD: đường viền).
2. **Tổ hợp thông tin chéo từ nhiều Kênh** (Cross-channel Mixing).

Chi phí tính toán khổng lồ:

$$K^2 \times C_{\text{in}} \times C_{\text{out}}$$



Giả thuyết mới: Đặc trưng Không gian và Đặc trưng Kênh có tính độc lập cao. Tại sao không tách chúng ra?

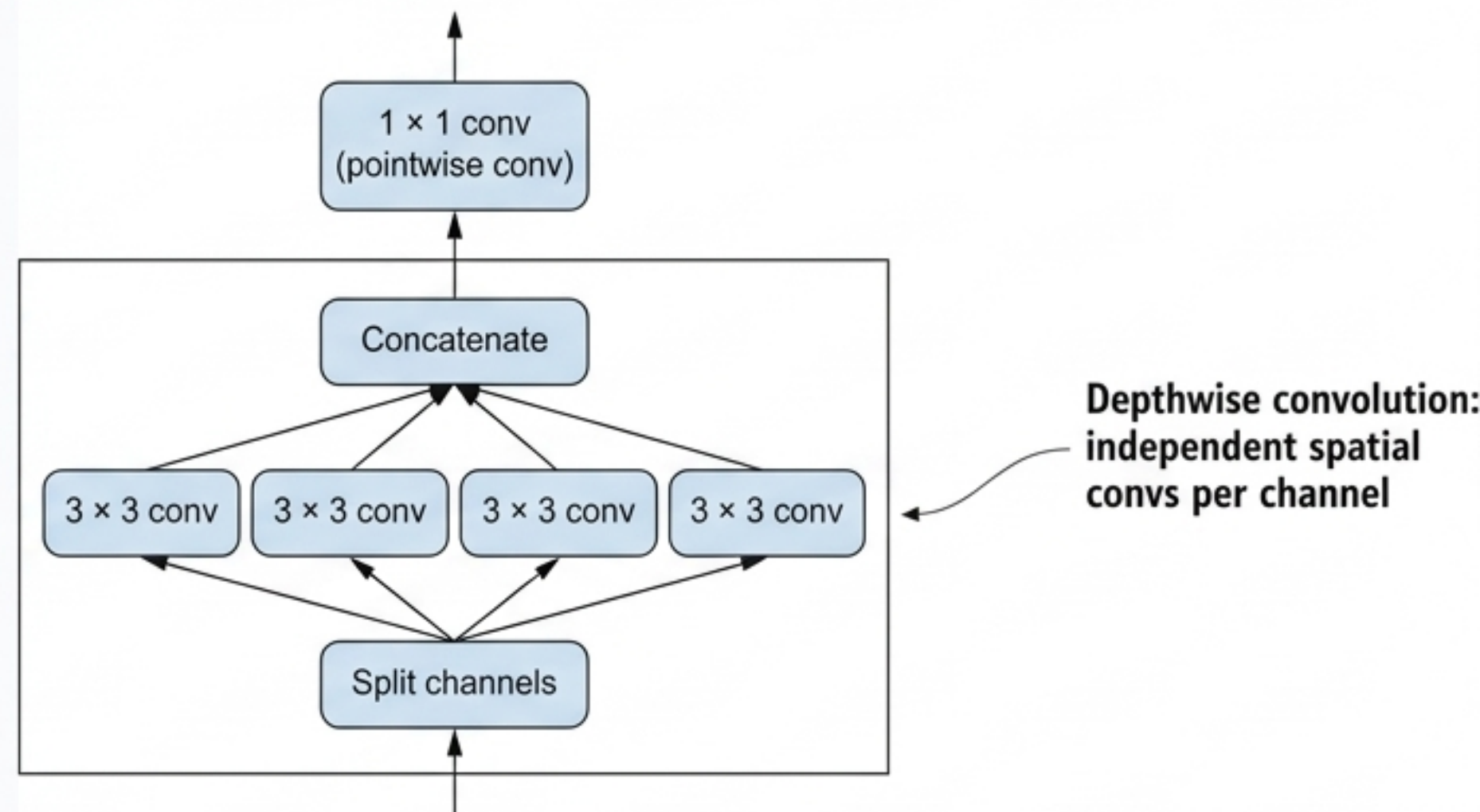
Tích chập Tách rời (Depthwise Separable Convolution)

Tách phép tính thành 2 pha độc lập:

1. **Depthwise Conv:** Quét không gian trên từng Kênh một cách độc lập (Không trộn lẫn).
2. **Pointwise Conv (1×1):** Nhào trộn thông tin giữa các Kênh tại từng điểm ảnh.

Tham số giảm mạnh xuống:

$$K^2 \times C_{\text{in}} + C_{\text{in}} \times C_{\text{out}}$$



Hình 9.4: Tích chập Tách rời chiều sâu

Phân tích Toán học về Hiệu suất

Giả sử: Kernel 3×3, Đầu vào 64 Channels, Đầu ra 64 Channels.

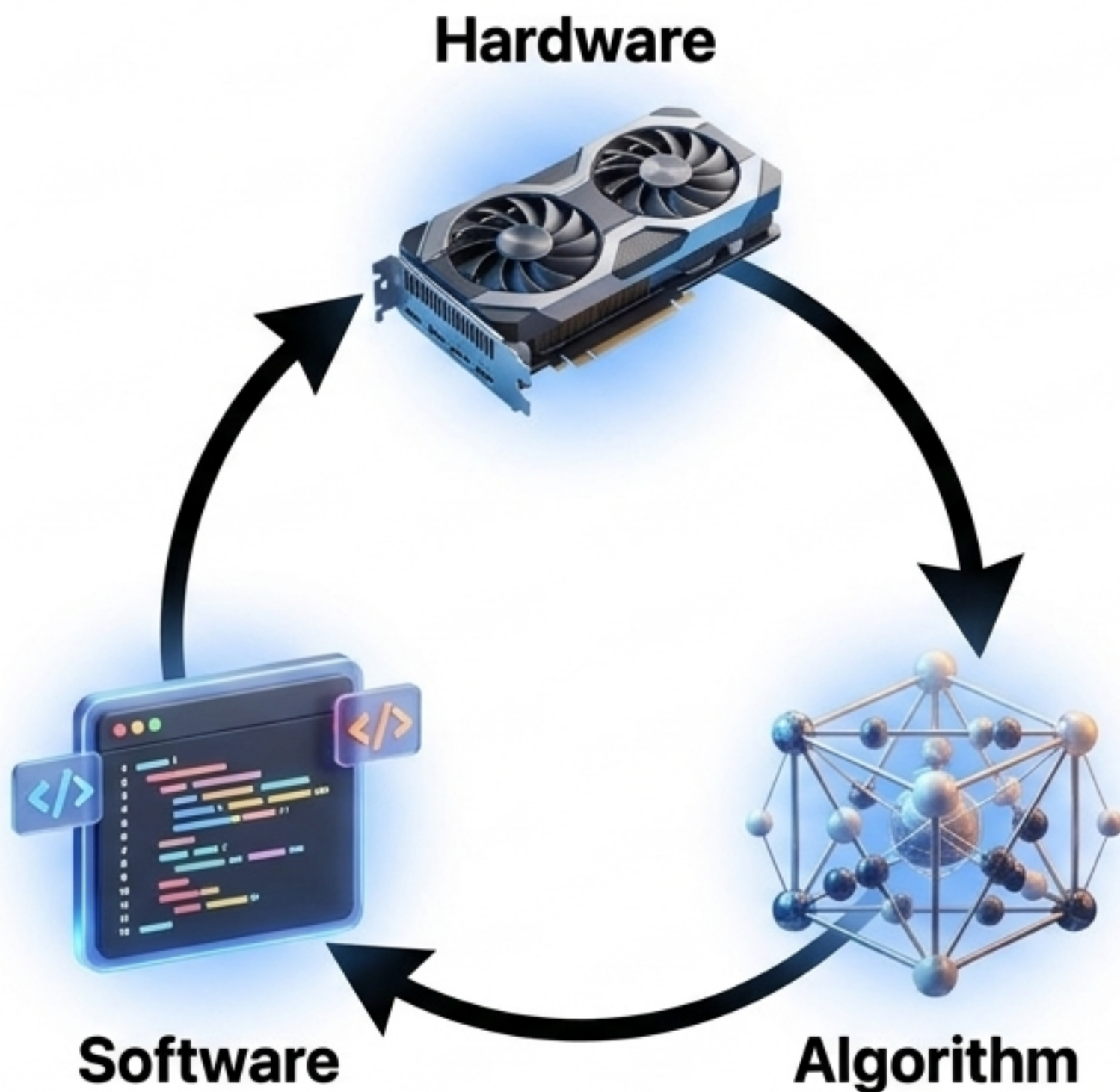
	Kiến trúc	Công thức	Tổng số tham số
1	Tích chập chuẩn (Conv2D)	$3^2 \times 64 \times 64$	36,864
2	Tích chập Tách rời (SeparableConv2D)	$(3^2 \times 64) + (64 \times 64)$	4,672



Insight

Tiết kiệm gần 90% bộ nhớ và năng lực tính toán. Mô hình hội tụ nhanh hơn, chống Overfitting tốt hơn do ít tham số dư thừa.

Sự tiến hóa đồng thời: Phần cứng & Phần mềm



- **Insight 1:**
Nghịch lý: Dù giảm 90% tham số, SeparableConv2D trên thực tế GPU chỉ chạy tương đương Conv2D thông thường.
- **Insight 2:**
Nguyên nhân (NVIDIA cuDNN): Lớp Conv2D tiêu chuẩn đã được tối ưu vi mô (micro-optimized) bằng hợp ngữ (Assembly) qua hàng thập kỷ để đạt tốc độ Exaflops.
- **Rào cản AI:** Các thuật toán mới dù ưu việt về toán học nhưng không được phần cứng thiết kế chuyên biệt sẽ khó phát huy tốc độ. Đây là rào cản ngăn chặn các đột phá đi chệch khỏi lối mòn Backpropagation.

Lắp ráp mô hình Mini-Xception (Phần 1)



```
inputs = keras.Input(shape=(180, 180, 3))
x = layers.Rescaling(1.0 / 255)(inputs)
x = layers.Rescaling(1.0 / 255)(inputs)
x = layers.Conv2D(filters=32, kernel_size=5, use_bias=False)(x)
for size in [32, 64, 128, 256, 512]:
    residual = x
    x = layers.BatchNormalization()(x)
    x = layers.Activation("relu")(x)
    x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)
```

1

Lớp đầu tiên PHẢI là Conv2D thường. Vì các kênh RGB của ảnh tự nhiên có độ tương quan cực cao, giả thuyết "Tách rời" không hiệu quả ở bước này!

2

Ứng dụng nguyên tắc chuẩn bị cho BatchNormalization.

2

Ứng dụng nguyên tắc chuẩn bị cho BatchNormalization.

Lắp ráp mô hình Mini-Xception (Phần 2)

```
x = layers.BatchNormalization()(x)
x = layers.Activation("relu")(x)
x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)
x = layers.MaxPooling2D(3, strides=2, padding="same")(x)
residual = layers.Conv2D(size, 1, strides=2, padding="same", use_bias=False)(residual)
x = layers.add([x, residual])
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
```

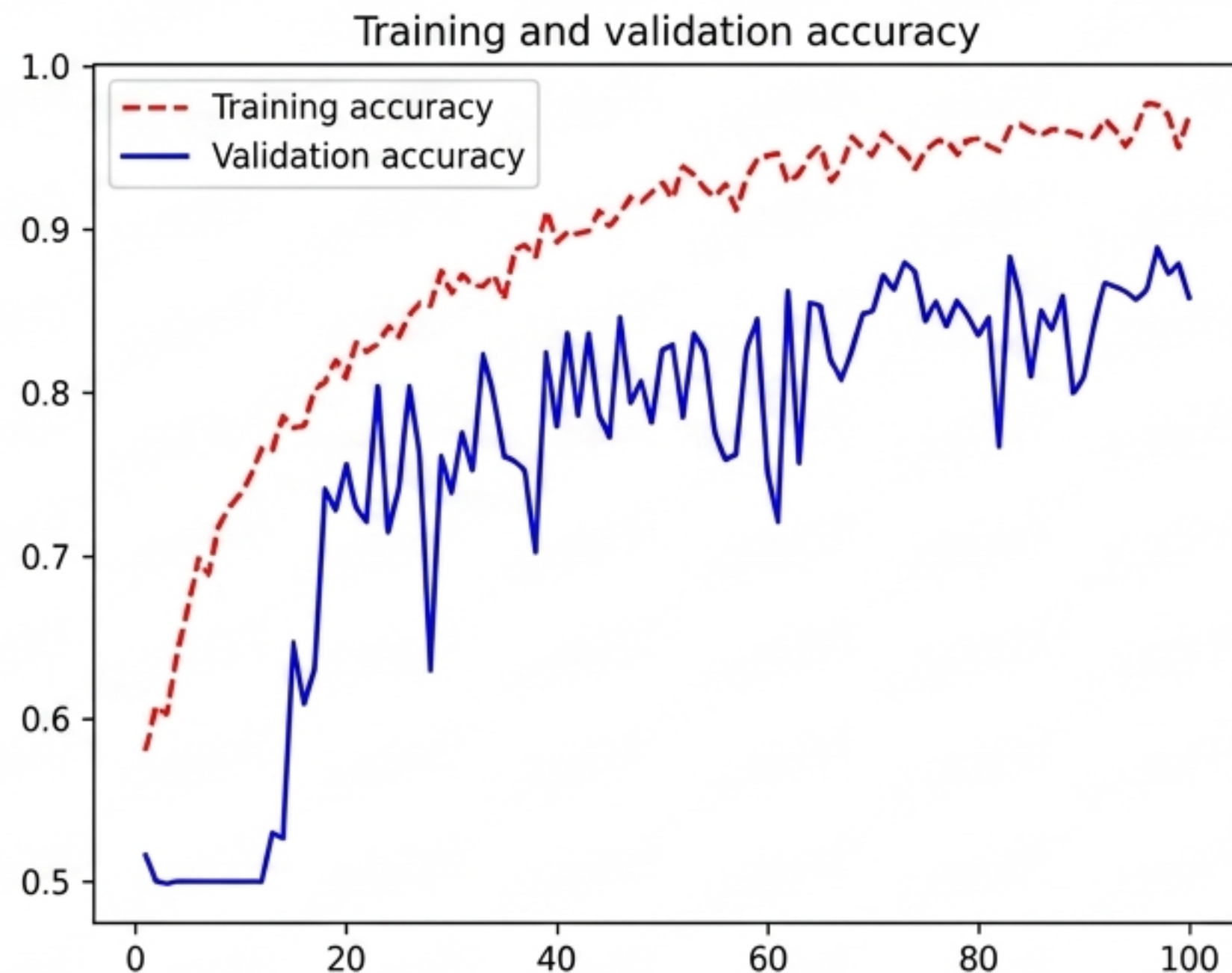


Kích thước
mạng cực nhẹ: Chỉ
721,857 tham số
so với 1.5 triệu
tham số của cấu
trúc CNN
cơ bản cũ!

Đánh giá Độ chính xác (Accuracy)

90.8% (Mini-Xception)
vs 83.9% (Basic CNN)

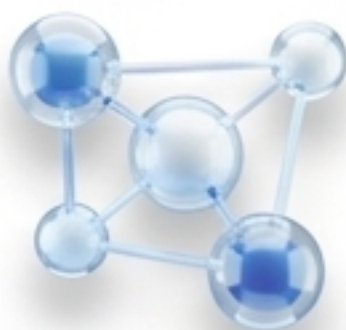
Nhờ cấu trúc siêu ưu việt tự thiết kế (không dùng Transfer Learning), mô hình đạt độ chính xác **90.8%** trên tập Chó/Mèo. Thiết kế kiến trúc chuẩn mực (Architecture Best Practices) mang lại bước nhảy vọt về hiệu năng.



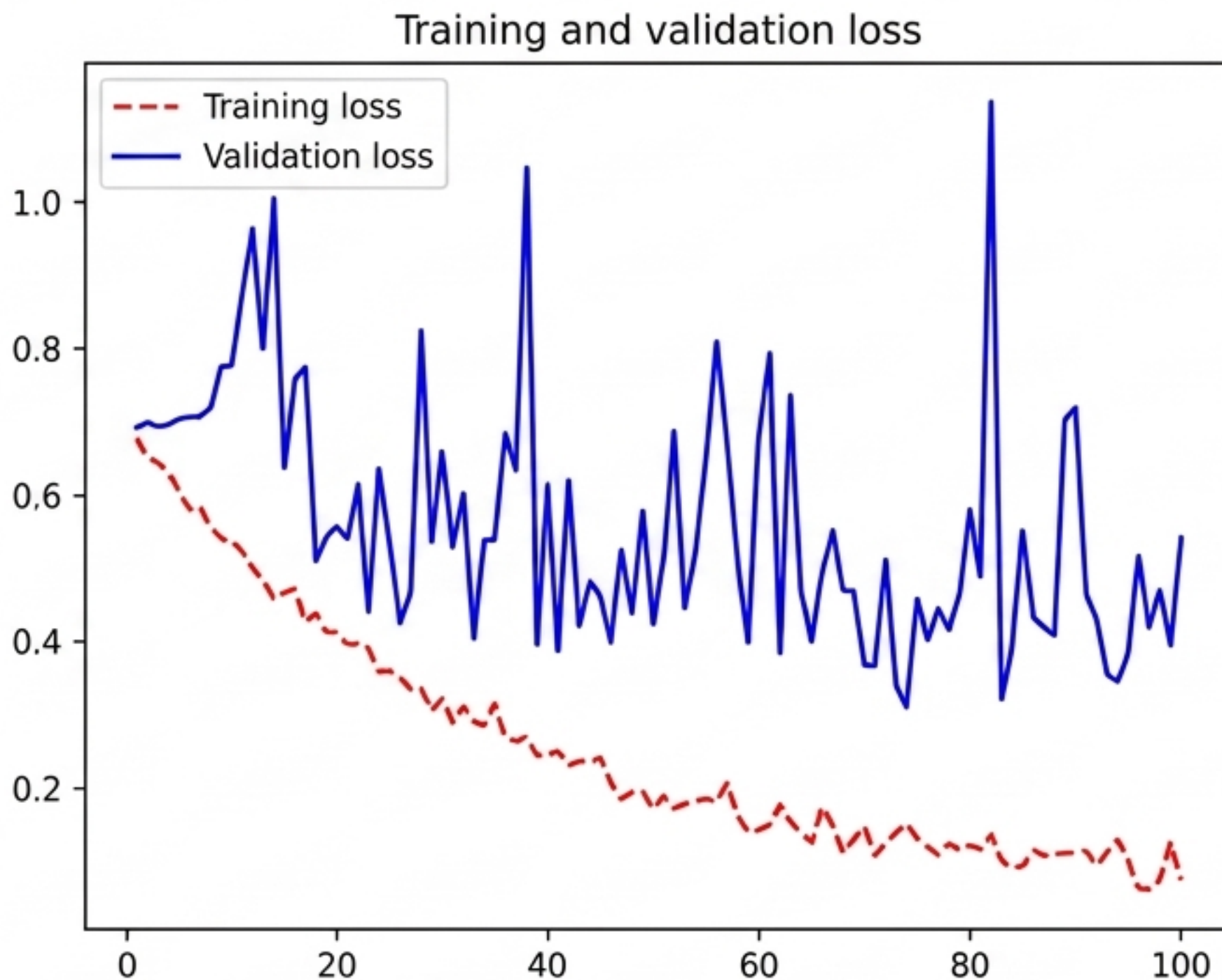
Hình 9.5: Accuracy của Mini-Xception

Đánh giá Mất mát (Loss)

Đường Validation Loss đi xuống ổn định và mượt mà hơn rất nhiều.



Sự hiện diện của BatchNormalization ở mọi ngõ ngách giúp thuật toán hội tụ cực nhanh và ổn định. Dạng kiến trúc lặp này (Xception) hiện là tiêu chuẩn công nghiệp (Industry Standard) trên thư viện Keras.



Hình 9.6: Loss của Mini-Xception

Giao thời Công nghệ: Beyond Convolution



Đế chế ConvNets

Mạng Tích chập thống trị
độc tôn lĩnh vực Thị giác
Máy tính nướng Thị giác
Máy tính suốt từ thập niên
2010.

Vs

Ý tưởng táo bạo:
Chuyện gì sẽ xảy ra
nếu ta coi một bức ảnh
giống như “Một câu
văn”, và các bộ phận
ảnh là “Các từ vựng”?



Kẻ thách thức - ViTs

Vision Transformers sinh
Vision Transformers sinh
ra năm 2017 dành riêng
cho Xử lý Ngôn ngữ Tự
nhiên (NLP - dịch thuật).

Cơ chế hoạt động & Sức mạnh Self-Attention

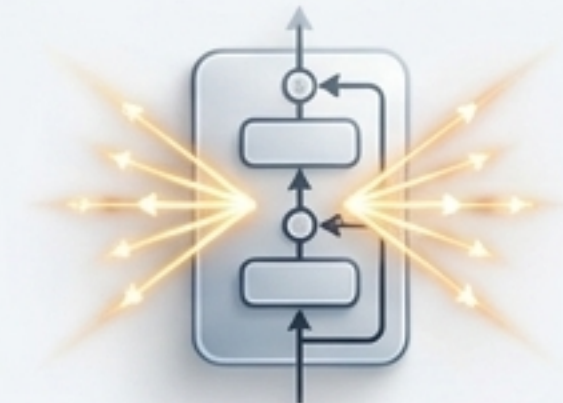
Bác bỏ hoàn toàn Conv2D.
Chặt ảnh gốc thành lưới các **mảnh ghép** (VD: 16×16 pixel).



Trải phẳng mỗi mảnh ghép thành một **Vector 1 chiều** (Biến nó thành một 'Từ vựng').



Bơm chuỗi **Vector** vào cơ chế **Self-Attention**.



Liên kết cự ly xa (Long-range relationships): Không giống ConvNet bị giới hạn bởi Kernel 3×3, ViT so sánh mọi Patch với mọi Patch khác cùng lúc. Nó hiểu ngay lập tức quả bóng ở góc trái liên quan mật thiết đến con chó đang chạy ở góc phải!

Hạn chế của ViTs & Kết luận Thực tiễn

Vision Transformers (ViTs)

Nhu cầu Dữ liệu

Cần lượng dữ liệu khổng lồ (Lớn hơn cả ImageNet) do như một tờ giấy trắng tự học không gian.

Hạ tầng Phần cứng

Mô hình cực nặng, đòi hỏi Siêu máy tính (Clusters of TPUs/GPUs).

ConvNets

Tận dụng 'Kiến thức ưu tiên 2D' (biết các pixel cạnh nhau có liên quan). Ít dữ liệu hơn.

Hoạt động tốt, đào tạo nhanh trên Standard GPUs.



Mặc dù ViTs là tương lai, nhưng đối với các bài toán doanh nghiệp thông thường với lượng dữ liệu có giới hạn, ConvNets vẫn là sự lựa chọn hoàn hảo nhất.